

---

# Energy Stock Price Forecasting

Using LSTM Neural Networks

---

Aarya Parekh

B.Tech Energy Engineering, IIT Bombay

August 2026

Self Project — Quantitative Finance & Deep Learning

# Table of Contents

1. Project Overview
2. Motivation & Objectives
3. Dataset Description
4. Methodology
  - 4.1 Exploratory Data Analysis
  - 4.2 Feature Engineering
  - 4.3 Baseline Models
  - 4.4 LSTM Architecture
  - 4.5 Backtesting & Trading Strategy
5. Results & Analysis
6. Technical Skills Demonstrated
7. Key Learnings
8. Future Scope

# 1. Project Overview

This project develops a deep learning-based stock price forecasting system for the Indian energy sector. Using 5 years of historical NTPC stock data (2019-2026), the system engineers 17 technical indicators and compares three machine learning models — Linear Regression, Random Forest, and LSTM neural network — to predict next-day closing prices.

The project goes beyond prediction accuracy by implementing a complete backtesting framework that simulates a trading strategy based on model predictions, evaluating it against a buy-and-hold benchmark using industry-standard metrics including the Sharpe ratio, maximum drawdown, and directional accuracy.

| Attribute   | Details                                                       |
|-------------|---------------------------------------------------------------|
| Asset       | NTPC Ltd (NSE: NTPC.NS)                                       |
| Data Period | August 2019 — August 2026 (~1,250 trading days)               |
| Features    | 17 technical indicators (trend, momentum, volatility, volume) |
| Models      | Linear Regression, Random Forest, LSTM                        |
| Framework   | PyTorch (LSTM), scikit-learn (baselines)                      |
| Evaluation  | RMSE, MAE, Directional Accuracy, Sharpe Ratio                 |

## 2. Motivation & Objectives

### Why this project?

The Indian energy sector is undergoing a massive transformation — from traditional fossil fuels to renewables, with government policies driving rapid changes in valuations. Energy stocks exhibit unique characteristics: sensitivity to crude oil prices, policy announcements, seasonal demand patterns, and global geopolitical events. This makes them an interesting and challenging target for time-series forecasting.

### Objectives

- Build a complete ML pipeline from raw data to trading strategy evaluation
- Compare traditional ML baselines against deep learning (LSTM) for time-series forecasting
- Engineer domain-specific features using established technical analysis indicators
- Implement a rigorous backtesting framework with proper train/validation/test splits
- Demonstrate understanding of financial metrics (Sharpe ratio, drawdown, directional accuracy)
- Produce a portfolio-ready project with clean code, visualizations, and documentation

### 3. Dataset Description

The dataset comprises daily OHLCV (Open, High, Low, Close, Volume) data for NTPC Ltd, downloaded via the yfinance Python library from Yahoo Finance. NTPC is India's largest power generation company and a key constituent of the NIFTY Energy Index, making it representative of the broader energy sector.

| Column | Description                         | Use in Project                   |
|--------|-------------------------------------|----------------------------------|
| Open   | Price at market open (9:15 AM IST)  | Price range calculation          |
| High   | Highest price during the day        | ATR, Stochastic, Bollinger Bands |
| Low    | Lowest price during the day         | ATR, Stochastic, Bollinger Bands |
| Close  | Price at market close (3:30 PM IST) | All indicators + target variable |
| Volume | Total shares traded                 | OBV, volume ratio                |

## 4. Methodology

### 4.1 Exploratory Data Analysis

Six visualizations were produced to understand the data characteristics before modeling:

- **Price History:** 5-year closing price trend revealing overall market trajectory, including the COVID-19 crash (March 2020) and subsequent recovery.
- **Daily Returns:** Percentage change distribution showing volatility clustering — calm periods followed by bursts of high volatility.
- **Rolling Volatility:** 30-day annualized volatility highlighting regime changes and crisis periods.
- **Returns Distribution:** Histogram with KDE overlay confirming non-normal returns with fat tails (excess kurtosis) and slight negative skew — extreme losses are more frequent than a normal distribution predicts.
- **Monthly Heatmap:** Year-by-month returns grid revealing seasonal patterns in energy stock performance.
- **ACF/PACF:** Autocorrelation analysis showing weak but statistically significant serial correlation in returns — justifying the use of sequence models like LSTM.

The Augmented Dickey-Fuller (ADF) test confirmed that raw prices are non-stationary ( $p > 0.05$ ) while daily returns are stationary ( $p < 0.05$ ), validating the standard approach of modeling returns or next-day prices rather than raw price levels.

### 4.2 Feature Engineering

17 technical indicators were engineered from raw OHLCV data using the `ta` (Technical Analysis) Python library, organized into four categories:

| Category       | Features                                    | Purpose                                      |
|----------------|---------------------------------------------|----------------------------------------------|
| Trend (6)      | SMA 20/50/200, EMA 12/26, MACD              | Capture price direction and momentum shifts  |
| Momentum (2)   | RSI, Stochastic K                           | Identify overbought/oversold conditions      |
| Volatility (3) | Bollinger Bands (upper/lower), ATR          | Measure price uncertainty and range          |
| Volume (1)     | On-Balance Volume (OBV)                     | Confirm trends with trading activity         |
| Custom (5)     | Returns (1d/5d/20d), Vol ratio, Price range | Multi-horizon momentum and intraday activity |

The target variable is the next-day closing price, created by shifting the Close column by one row. A binary Direction variable (1=up, 0=down) was also created for directional accuracy evaluation.

## **Data Split**

The dataset was split chronologically (not randomly) to prevent data leakage — a critical requirement for time-series problems. The split ratios are 70% train, 15% validation, 15% test. Features were standardized using StandardScaler fitted only on training data.

### 4.3 Baseline Models

**Linear Regression** serves as the simplest baseline, fitting a linear combination of all 17 features to predict price. Despite its simplicity, it establishes a minimum performance threshold.

**Random Forest Regressor** (200 trees, max depth 10) captures non-linear feature interactions that Linear Regression misses. It also provides feature importance rankings, revealing which technical indicators carry the most predictive signal. This model typically outperforms linear methods on financial data due to the inherently non-linear nature of market dynamics.

### 4.4 LSTM Architecture

The primary model is a 2-layer Long Short-Term Memory (LSTM) network implemented in PyTorch. LSTMs are specifically designed for sequential data — they maintain an internal memory state that captures temporal dependencies across time steps, making them well-suited for financial time series where patterns unfold over multiple days.

| Component         | Configuration                              |
|-------------------|--------------------------------------------|
| Input             | 30-day sliding window of 17 features       |
| LSTM Layer 1      | 64 hidden units, dropout 0.2               |
| LSTM Layer 2      | 64 hidden units, dropout 0.2               |
| Dense Layer 1     | 64 → 32 units, ReLU activation             |
| Dropout           | 0.1                                        |
| Output Layer      | 32 → 1 (predicted price)                   |
| Loss Function     | Mean Squared Error (MSE)                   |
| Optimizer         | Adam (lr=0.001)                            |
| LR Scheduler      | ReduceLROnPlateau (patience=5, factor=0.5) |
| Early Stopping    | Patience=10 on validation loss             |
| Gradient Clipping | Max norm = 1.0                             |

### 4.5 Backtesting & Trading Strategy

A simple long-only trading strategy was implemented: if the LSTM predicts tomorrow's price will be higher than today's close, take a long position; otherwise, stay flat (no position). This strategy was evaluated against a buy-and-hold benchmark using the following metrics:

- **Sharpe Ratio:** Risk-adjusted return (excess return / volatility). Above 1.0 is considered good, above 2.0 is excellent.
- **Maximum Drawdown:** Worst peak-to-trough portfolio loss — measures downside risk.
- **Cumulative Return:** Total portfolio growth over the test period.
- **Win Rate:** Percentage of trading days where the position was profitable.
- **Directional Accuracy:** How often the model correctly predicted up vs down movement.

# 5. Results & Analysis

Model performance was evaluated on the held-out test set (the final 15% of data, representing the most recent time period). Results will be populated after all models have been trained.

| Model             | RMSE           | MAE            | Dir. Accuracy  |
|-------------------|----------------|----------------|----------------|
| Linear Regression | [to be filled] | [to be filled] | [to be filled] |
| Random Forest     | [to be filled] | [to be filled] | [to be filled] |
| LSTM              | [to be filled] | [to be filled] | [to be filled] |

\* Results table will be updated with actual values after LSTM training and backtesting.

The six EDA visualizations and three model evaluation plots (training curves, actual vs predicted, cumulative returns) provide comprehensive visual evidence of the analysis. The feature importance plot from the Random Forest model reveals which technical indicators carry the strongest predictive signal for NTPC stock price movements.

## 6. Technical Skills Demonstrated

| Domain              | Skills                                                                                           |
|---------------------|--------------------------------------------------------------------------------------------------|
| Programming         | Python, object-oriented design, clean code practices                                             |
| Data Engineering    | pandas, numpy, data cleaning, feature engineering, chronological splitting                       |
| Machine Learning    | scikit-learn, StandardScaler, Linear Regression, Random Forest, hyperparameter tuning            |
| Deep Learning       | PyTorch, LSTM networks, early stopping, learning rate scheduling, gradient clipping              |
| Financial Analysis  | Technical indicators (SMA, EMA, MACD, RSI, Bollinger Bands, ATR, OBV), backtesting, Sharpe ratio |
| Visualization       | matplotlib, seaborn, time-series plots, heatmaps, ACF/PACF                                       |
| Statistical Testing | ADF stationarity test, returns distribution analysis, autocorrelation                            |
| Version Control     | Git, GitHub                                                                                      |

## 7. Key Learnings

- **Time-series splits are non-negotiable:** Random shuffling leaks future data into training, producing artificially inflated accuracy that collapses on real unseen data.
- **Feature engineering matters more than model complexity:** Well-chosen technical indicators can make even a Linear Regression competitive. The model is only as good as the features it sees.
- **Baselines prevent over-claiming:** Without a Linear Regression baseline, you can't prove the LSTM adds value. If LSTM beats baselines by only 1%, the added complexity may not be justified.
- **Financial returns have fat tails:** Normal distribution assumptions break down — extreme moves happen far more often than expected, which is why models trained on MSE loss can underperform during volatile periods.
- **Directional accuracy matters more than RMSE for trading:** A model that's off by 50 rupees but always gets the direction right is more profitable than one that's off by 10 rupees with random direction.

## 8. Future Scope

- Incorporate macroeconomic features (crude oil prices, interest rates, FII flows)
- Extend to multiple energy stocks with a portfolio-level strategy
- Experiment with Transformer-based architectures (e.g., Temporal Fusion Transformer)
- Add sentiment analysis from financial news and social media
- Implement more sophisticated position sizing (Kelly criterion, risk parity)
- Deploy as a real-time prediction dashboard with live data feeds

---

This project was completed as a self-directed initiative to develop practical skills in quantitative finance and deep learning. All code, data processing, and analysis were performed independently.